ADVANCED DATA MINING · FINAL PROJECT

Self-Supervised Anomaly Detection in GPS Taxi Trajectories

TS2Vec embeddings vs. hand-crafted 8 features for unsupervised anomaly detection in the Porto Taxi dataset (ECML-PKDD 2015), evaluated via synthetic anomaly injection across four corruption types.

[Hubert Stolarz - Karol Kulig] · *ADM 2026*

Dataset

Porto Taxi · ECML-PKDD 2015 Trajectory Challenge

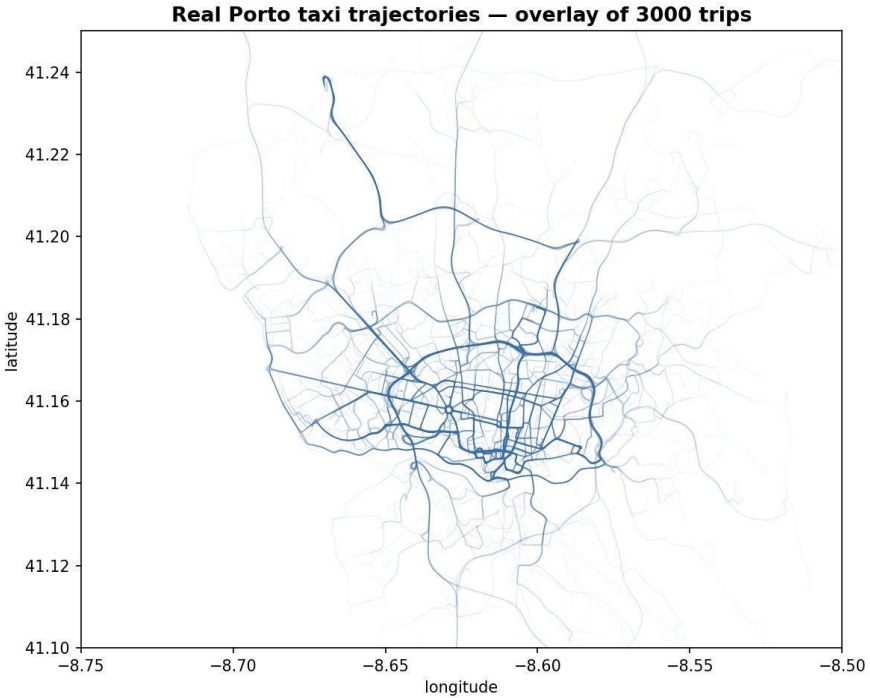
1.7 M trips total

442 taxis

1 year Jul 2013 – Jun 2014

15 s GPS sampling rate

54,573 trips after cleaning



The problem

Detect anomalous taxi trips in GPS trajectory data, without ground-truth labels.

Detours

Trips much longer than the direct route between origin and destination.

GPS errors

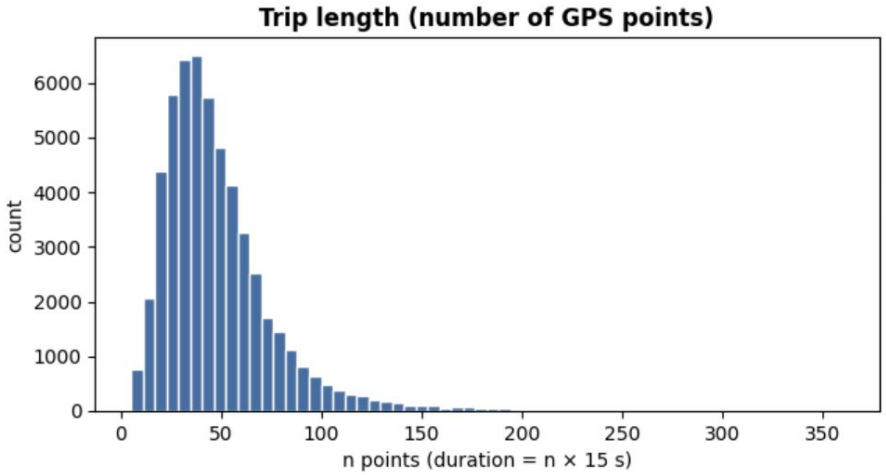
Recording glitches: impossible jumps, unrealistic speeds, point teleports.

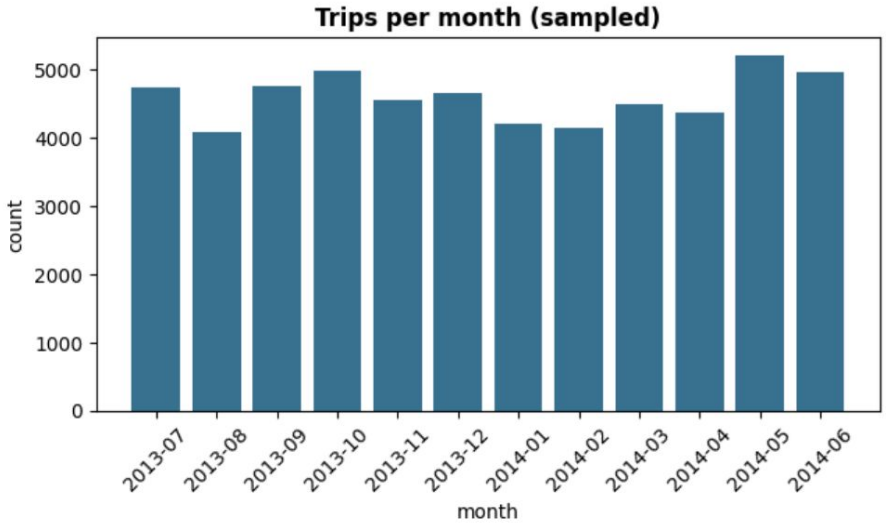
Unusual routes

Trajectories with structurally rare shapes or movement patterns.

Challenge: no ground-truth anomaly labels exist — we must create our own evaluation.

Dataset






```
Trip length percentiles (n_points): 5% = 16, median = 42, 95% = 102
In seconds:                         5% = 240s, median = 630s, 95% = 1530s
```

Approach

Compare learned (TS2Vec) and interpretable (hand-crafted) trip representations under the same anomaly scorer.

TS2Vec (learned)

3 channels per trip

Δlon, Δlat, speed (in meters / $m \cdot s^{-1}$)

Padded to 200 timesteps (NaN if trip shorter)

Trained 5 epochs on 10k random trips

Mean-pool ⟶ 320-dim trip embedding

Baseline (hand-crafted)

8 hand-crafted features per trip

length, n_points, straight-line

detour ratio

max / mean / std speed

bounding box diagonal

All in meters / $m \cdot s^{-1}$

Same scorer for both: Isolation Forest (200 trees, contamination='auto', identical random seed)

Implementation pipeline

Four notebooks executed in sequence — Google Colab T4 GPU runtime, ~30 min end-to-end.

01 · Load & clean

Load 1.7M trips from Kaggle (~1.9 GB)
Filter MISSING_DATA, empty polylines
Drop trips < 5 or > 360 points (meter-left-on)
Random sample every 30th row across 12 months
Output: 54,573 clean trips

02 · Baseline + injection

Extract 8 hand-crafted features per trip
Inject 4 anomaly types × 200 = 800 anomalies
Fit IsolationForest on standardized features
Compute per-type ROC-AUC + PR-AUC

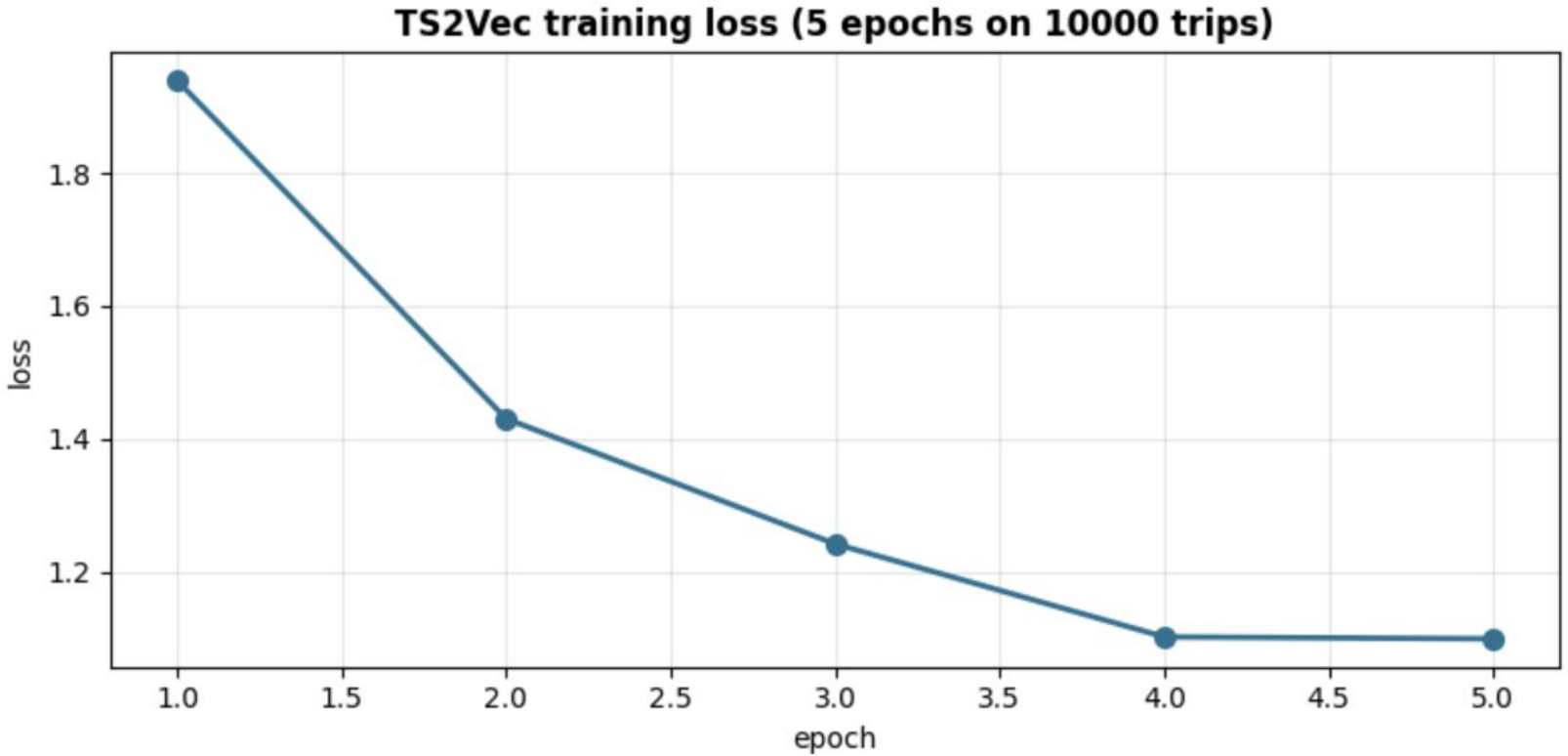
03 · TS2Vec training

Convert trips to (Δlon, Δlat, speed)
Pad to 200 timesteps with NaN
Train TS2Vec 5 epochs on 10k random subset
Encode all 54k trips ⟶ mean-pool to 320-dim
IsolationForest on embeddings ⟶ scores

04 · Maps & analysis

Top-12 by each method (injected + real)
Disagreement analysis on real trips
Per-trip rank scatter, score distributions
5 figures + 1 final comparison table

TS2Vec training



How we inject anomalies

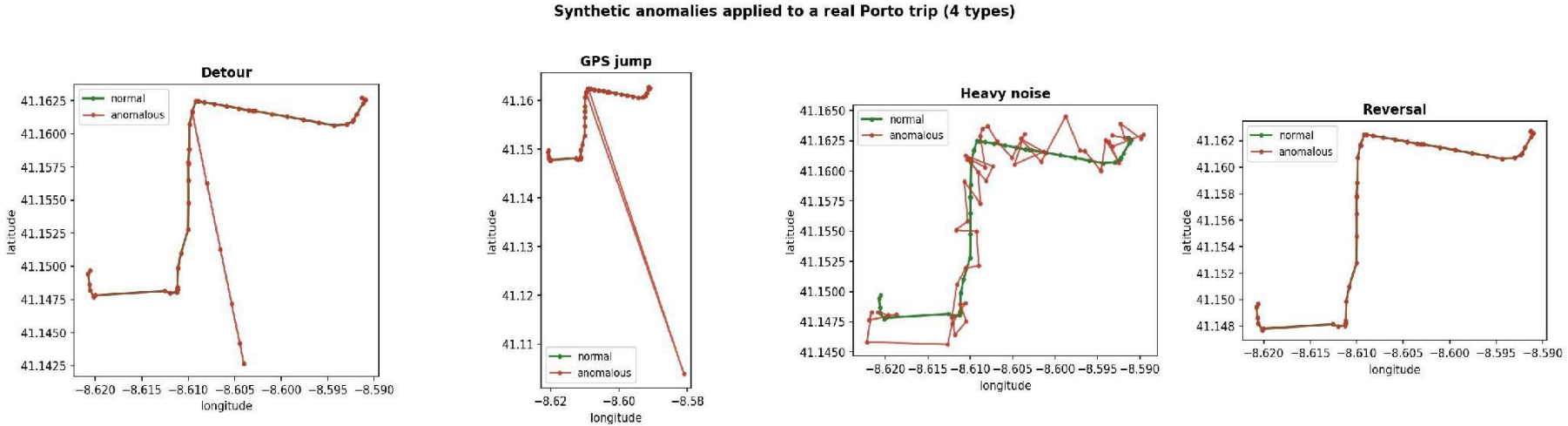
Each injection modifies a real trip in a controlled way, the label is known, so we can measure detection.

Type	How it's injected	What it changes	Which method should catch it
Detour	Insert 12-point loop ~2 km off path	↑ length ↑ detour_ratio	Baseline (detour_ratio)
GPS jump	Teleport one point ~3 km away	↑↑ max_speed (single huge segment)	Baseline (max_speed)
Noise	Gaussian noise (~150 m) on every point	↑ speed_std (distributed corruption)	Both (TS2Vec more normally)
Reversal	Flip sequence in time pts[::-1]	Nothing in features (order-invariant)	TS2Vec only*

** Normally mean-pooling washes out sequence ordering, neither method catches reversal reliably. Attention-pooling would have worked.*

Evaluation: synthetic anomalies on real trips

Each injection applied to one real Porto trip — green = original, red = corrupted.



Detour

midpoint loop
~2 km bump

GPS jump

teleport one point
~3 km away

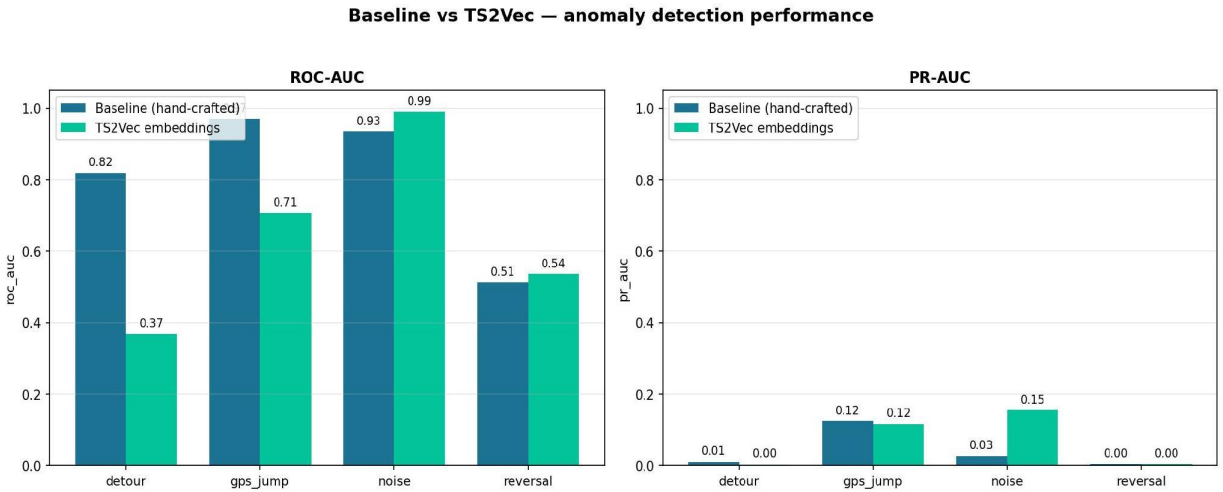
Heavy noise

Gaussian noise on
every point (~150 m)

Reversal

flip sequence in time
(invariant to baseline)

Results: ROC-AUC per anomaly type



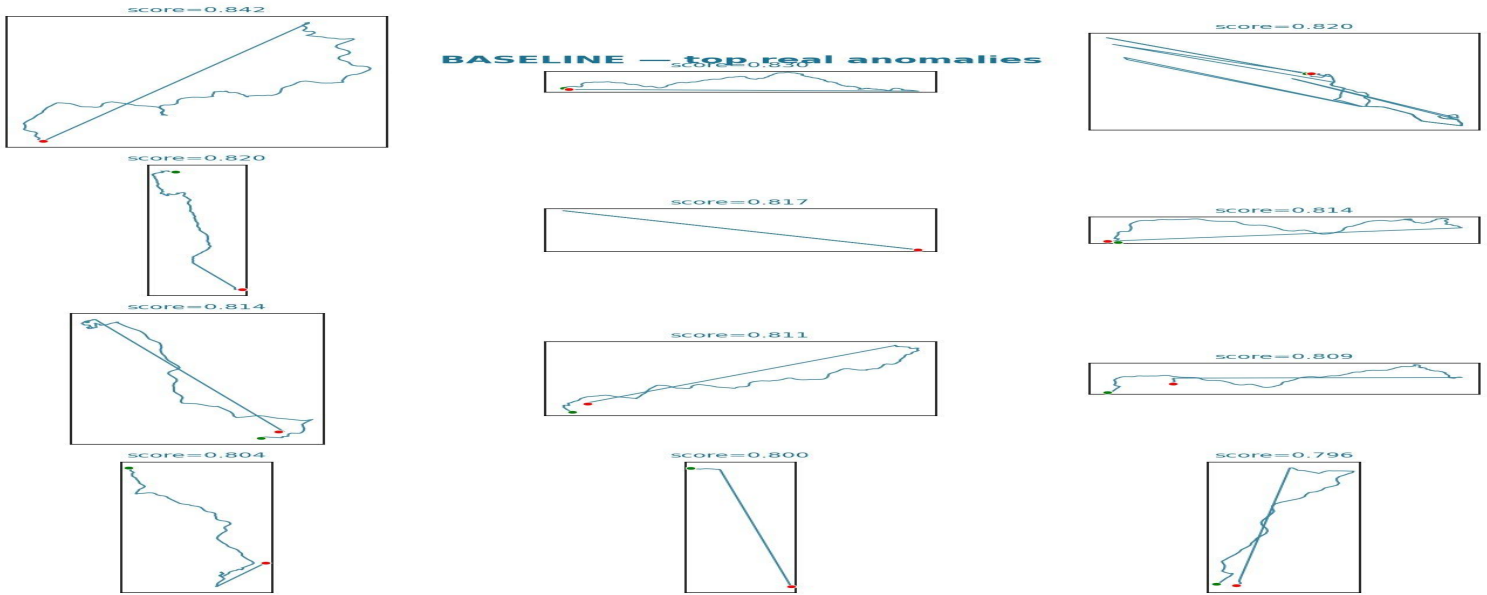
Per-type ROC-AUC

	Base	TS2Vec	
detour	0.819	0.349	**Base**
gps_jump	0.970	0.769	**Base**
noise	0.934	0.988	**TS2V**
reversal	0.512	0.552	**TS2V**

Baseline wins on geometric anomalies (detour, GPS jump). TS2Vec wins on noise (distributed corruption). Both ≈ random on reversal — sequence ordering is washed out by mean-pooling.

Baseline's top real anomalies

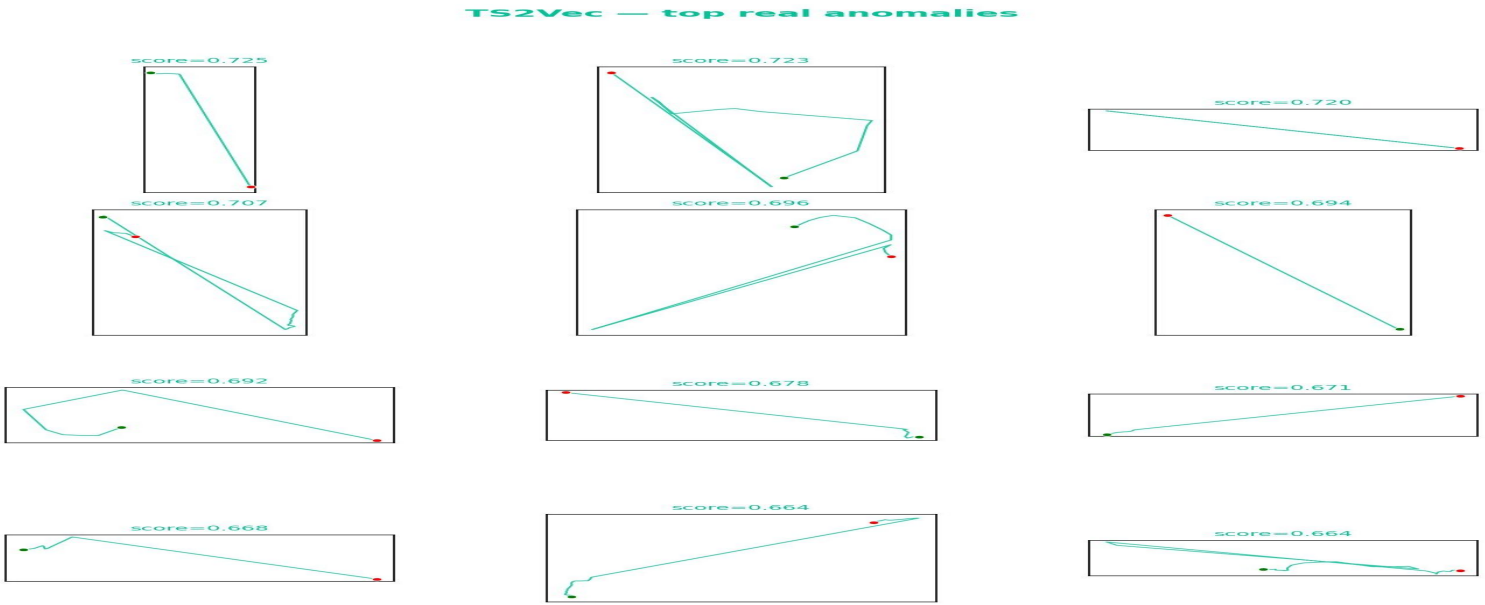
12 highest-scored uninjected trips by hand-crafted features.



Pattern: long, near-straight routes, likely airport runs or intercity fares. Baseline catches them via extreme length, max_speed, and bbox features.

TS2Vec's top real anomalies

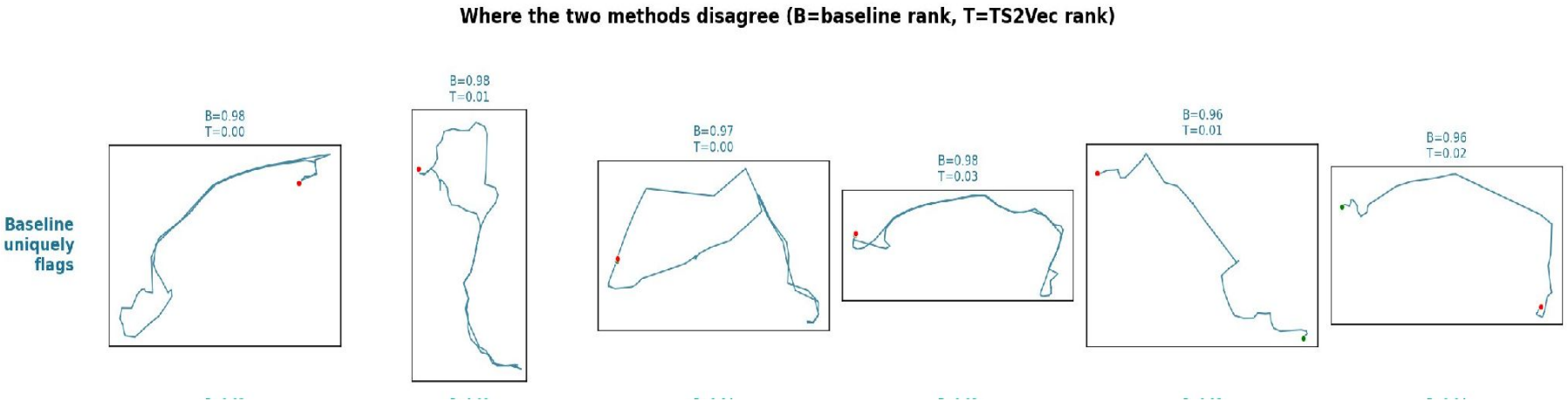
12 highest-scored uninjected trips by learned embeddings.



Pattern: structurally unusual shapes — zigzags, polygons, loops. TS2Vec captures shape complexity, not extreme magnitude.

Trips baseline catches but TS2Vec misses

Real trips where baseline rank ≈ 0.97 but TS2Vec rank ≈ 0.01.

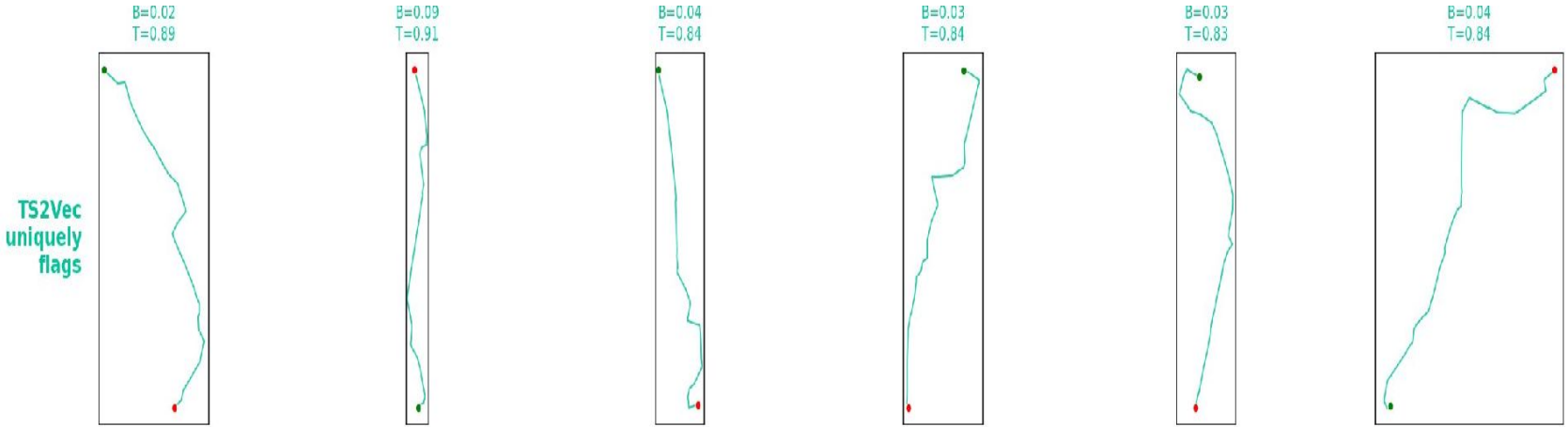


Baseline catches them because extreme geometric features (length, detour ratio, bounding box) push them to the top of the scalar distribution.

TS2Vec misses them — their local movement patterns look ordinary, so the embedding rates them as structurally normal.

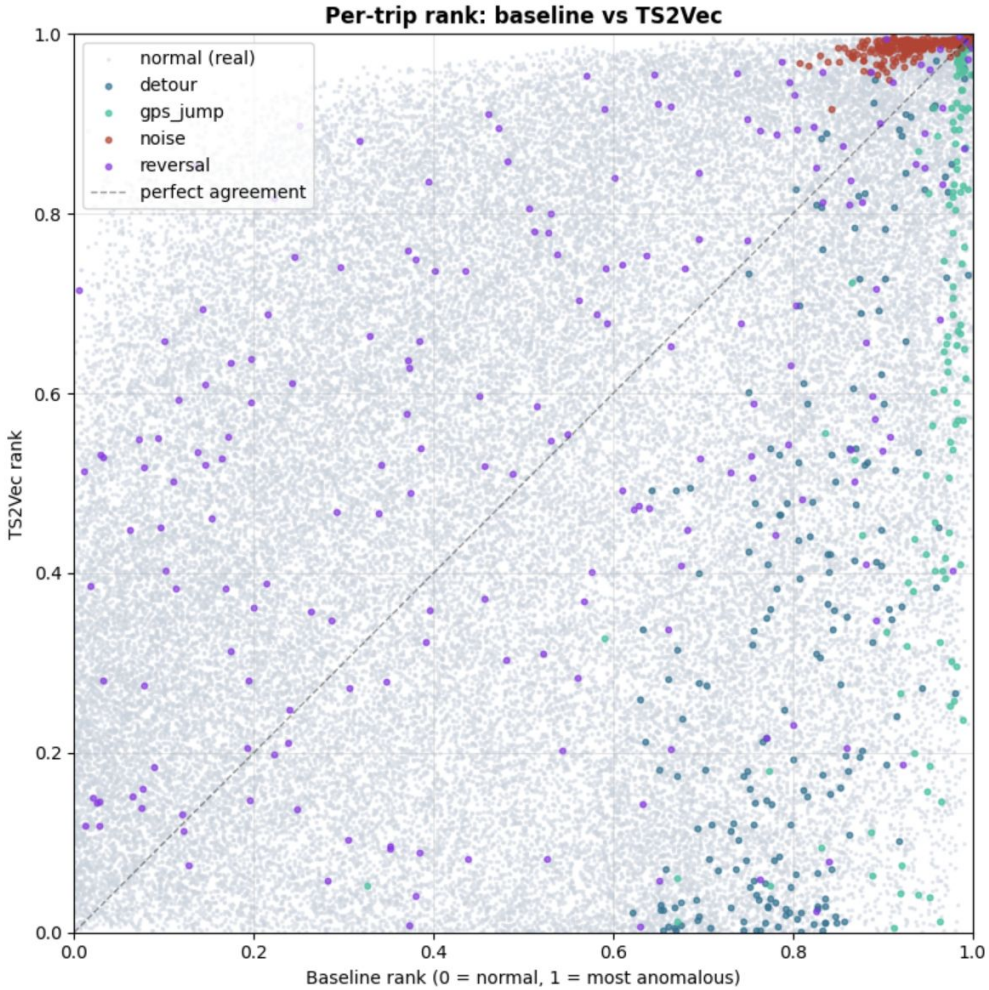
Trips TS2Vec catches but baseline misses

Real trips where TS2Vec rank ≈ 0.85 but baseline rank ≈ 0.03.



TS2Vec catches them because unusual local movement patterns (elongated curves, structurally rare shapes) move the embedding away from the dense cluster of normal trips.

Baseline misses them — scalar features (length, max_speed, bbox) rate them as ordinary, no extreme value stands out.


Per-trip rank: baseline vs TS2Vec
normal (real)
detour
gps_jump
noise
reversal
perfect agreement
TS2Vec rank
Baseline rank (0 = normal, 1 = most anomalous)
0.0
0.2
0.4
0.6
0.8
1.0

Conclusion

Learned and hand-crafted representations capture fundamentally different notions of anomaly.

- ✓ Per-type breakdown reveals strengths missed by global AUC
- ✓ Baseline excels on geometric anomalies (detour, GPS jump)
- ✓ TS2Vec captures distributed corruption (noise)
- ~ Both fail on reversal, mean-pooling washes out sequence ordering

Future work:

Sequence-sensitive pooling (last-token, attention); road-network matching for true detour detection; temporal context (time of day, day type).

Thanks for listening !